

Lecture 7: SQL - Part 3

Amit Kumar Dhar

August 13, 2025

Advanced Querying

Modified Schema for Today's Lecture

To illustrate certain concepts, we'll assume a slightly modified Students table:

Students(sid,sname,discip,registration_date)

sid	sname	discip	registration_date
101	Isha	CSE	2023-08-01
102	Parth	DSAI	2023-08-03
103	Juhi	CSE	2022-08-01
104	Suresh	MA	2023-09-01

The Courses and Enrolled tables remain the same.

Pattern Matching

The LIKE operator is used in a WHERE clause to search for a specified pattern in a column.

- % represents zero, one, or multiple characters.
- _ represents a single character.

Example: Find all students whose name contains 'li'.

```
SELECT sname, discip FROM Students  
WHERE sname LIKE '%a%';
```

Result:

sname	discip
Isha	CSE
Parth	DSAI

Handling NULL Values

SQL uses three-valued logic: TRUE, FALSE, and UNKNOWN.

- Any arithmetic or comparison with NULL results in NULL (which is treated as UNKNOWN).
- WHERE grade = NULL or WHERE grade <> NULL will not work as expected.

Use IS NULL or IS NOT NULL to check for null values.

Example: Find students who do not have a discip assigned.

```
SELECT sid, sname FROM Students  
WHERE discip IS NULL;
```

Date and Time

SQL supports various date/time data types (DATE, DATETIME, TIMESTAMP). SQLite stores them as text, which can be manipulated with functions.

Example: Find students who registered in the year 2023.

```
SELECT sname, registration_date FROM Students  
WHERE STRFTIME('%Y', registration_date) = '2023';
```

Result:

sname	registration_date
Isha	2023-08-01
Parth	2023-08-03
Suresh	2023-09-01

Date Operations

Date functions allow for powerful comparisons.

Example 1: Find students who registered in August 2023.

```
SELECT sname, registration_date FROM Students
WHERE registration_date
BETWEEN '2023-08-01' AND '2023-08-31';
```

Example 2: Find students who registered in the last 30 days from now.

```
SELECT sname, registration_date FROM Students
WHERE registration_date >= DATE('now', '-30 days');
```

Note: DATE('now') gives the current date.

Set Operation: EXCEPT

EXCEPT returns all rows from the first SELECT statement that are not returned by the second SELECT statement.

Example: Find the names of all 'CSE' students who are **not** enrolled in 'Databases' (CSL303).

```
-- First, get all CSE students
SELECT s.sname FROM Students s WHERE s.discip = 'CSE'
EXCEPT
-- Then, subtract those enrolled in CSL303
SELECT s.sname FROM Students s
JOIN Enrolled e ON s.sid = e.sid
WHERE e.cid = 'CSL303';
```


Subqueries

A subquery is a query nested inside another query.

Scalar Subquery: Returns a single value (one row, one column).

Example: Find students with a GPA higher than the average GPA.

```
SELECT sname, gpa FROM Students  
WHERE gpa > (SELECT AVG(gpa) FROM Students);
```

The inner query (SELECT AVG(gpa) ...) runs once and returns a single value.

Subqueries with Relations

Subqueries can return a full relation (multiple rows/columns).
Operators like IN, EXISTS, ANY, ALL are used.

Example with IN: Find the names of students enrolled in any 4-credit course.

```
SELECT sname FROM Students
WHERE sid IN (
    SELECT e.sid FROM Enrolled e JOIN Courses c ON e.cid = c.cid
    WHERE c.credits = 4
);
```

Subqueries with Relations

Subqueries can return a full relation (multiple rows/columns).

Operators like IN, EXISTS, ANY, ALL are used.

Example with EXISTS: Find discips that have at least one student.

```
SELECT DISTINCT discip FROM Students s1
WHERE EXISTS (
    SELECT 1 FROM Students s2
    WHERE s2.discip = s1.discip
);
```

Subqueries with ANY and ALL

ANY and ALL are used with comparison operators (=, <>, <, >, <=, >=).

Example with > ANY: Find students whose GPA is greater than at least one student in the 'DSAI' discip.

```
SELECT sname, gpa FROM Students
WHERE gpa > ANY (
    SELECT gpa FROM Students WHERE discip = 'DSAI'
);
```

This is equivalent to 'gpa > (SELECT MIN(gpa) ...)'

Subqueries with ANY and ALL

ANY and ALL are used with comparison operators (=, <>, <, >, <=, >=).

Example with > ALL: Find students whose GPA is greater than all students in the 'DSAI' discip.

```
SELECT sname, gpa FROM Students
WHERE gpa > ALL (
    SELECT gpa FROM Students WHERE discip = 'DSAI'
);
```

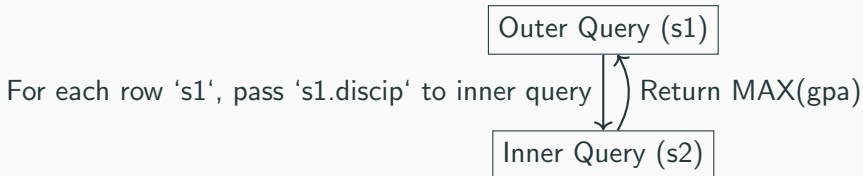
This is equivalent to 'gpa > (SELECT MAX(gpa) ...)'

Correlated Subqueries

A correlated subquery is a subquery that uses values from the outer query. It is evaluated once for each row processed by the outer query.

Example: Find students whose GPA is the highest in their discip.

```
SELECT s1.sname, s1.discip, s1.gpa FROM Students s1
WHERE s1.gpa = (
    SELECT MAX(s2.gpa) FROM Students s2
    WHERE s2.discip = s1.discip -- Correlation
);
```



DDL and DML

DML: Insertion

INSERT adds new rows to a table.

Variation 1: Inserting a single row.

```
INSERT INTO Students (sid, sname, discip, registration_date)
VALUES (201, 'Eve', 'Biology', '2024-01-15');
```

Variation 2: Inserting results from a query.

```
-- Create a new table first
CREATE TABLE CSE_Honors (
    student_id INTEGER,
    student_name TEXT
);
-- Insert all CSE students with GPA > 7.5
INSERT INTO CSE_Honors (student_id, student_name)
SELECT sid, sname FROM Students
WHERE discip = 'CSE' AND gpa > 7.5;
```


DML: Updation

UPDATE modifies existing records. **Always use a WHERE clause!**

Variation 1: Simple update.

```
UPDATE Students
SET discip = 'Computer Science'
WHERE discip = 'CSE';
```

Variation 2: Update based on another table.

```
-- Give a 5% GPA boost to students who got
-- an 'A' in Databases
UPDATE Students
SET gpa = gpa * 1.05
WHERE sid IN (
    SELECT sid FROM Enrolled
    WHERE cid = 'CSL303' AND grade = 'A'
);
```

DDL: Alter Table

ALTER TABLE modifies a table's structure. Support varies between SQL dialects.

Variation 1: Add a column.

```
ALTER TABLE Students  
ADD COLUMN phone_no TEXT;
```

Variation 2: Rename a table.

```
ALTER TABLE Students  
RENAME TO Student_Info;
```

Note: SQLite has limited support. Dropping columns or changing types is often done by creating a new table, copying data, and dropping the old one.

DDL: Foreign Key Constraints

A foreign key is a key used to link two tables together. It enforces referential integrity.

Example: The Enrolled table definition.

```
CREATE TABLE Enrolled (  
    sid INTEGER,  
    cid TEXT,  
    grade TEXT,  
    PRIMARY KEY (sid, cid),  
    FOREIGN KEY (sid) REFERENCES Students(sid)  
        ON DELETE CASCADE,  
    -- If a student is deleted, their enrollments are deleted  
    FOREIGN KEY (cid) REFERENCES Courses(cid)  
        ON UPDATE CASCADE  
    -- If a course cid is updated, it's updated here too  
);
```

Thank You!
Questions?